

AGENT-READY BUILD GUIDE

Autonomous AI Job Application Platform

Implementation Specification v1.0

DOCUMENT TYPE

Implementation Reference

BUILD TARGET

AI Coding Agent / Dev Team

STACK

Next.js · FastAPI · LangGraph

HORIZON

16 Weeks / 4 Phases

CLASSIFICATION: IMPLEMENTATION REFERENCE

Authored for Vikram Deshpande — Technical Delivery Lead & AI Solutions Architect

v1.0.0 • July 2026 • Companion to Architecture Document

Table of Contents

1. Implementation Overview	2	4. Phase 3 — Automation (Weeks 9–12)	
2. Phase 1 — Foundation (Weeks 1–4)	4	4.1 Job Board Integration	
2.1 Project Scaffolding	5	4.2 Application Submission	
2.2 Database Schema (Prisma)	6	4.3 Email Integration	
2.3 API Contract Specification	8	5. Phase 4 — Learning & Scale (Weeks 13–16)	
2.4 Authentication Setup	9	6. Configuration Specification	
3. Phase 2 — Agent Implementation (Weeks 5–8)		17. Testing Strategy	
3.1 Agent Framework Setup	11	8. Deployment Specification	
3.2 Individual Agent Specifications (20 Agents)	13	9. Monitoring & Observability	
3.3 Orchestration Graph	25	10. Quick Reference	

SECTION 01

Implementation Overview

System context, build order, environment requirements, and repository structure

1. Implementation Overview

This document is the **definitive build guide** for the Autonomous AI Job Application Platform — a 20-agent, multi-phase system purpose-built for Vikram Deshpande's Melbourne-based Technical Delivery Lead and AI Solutions Architect job search. It is a direct companion to the Architecture Document and translates every architectural decision into executable steps, code artifacts, and configuration files. An AI coding agent (Cursor, Aider, Devin) or a human development team can follow this guide sequentially from environment bootstrap to production deployment without requiring external clarification.

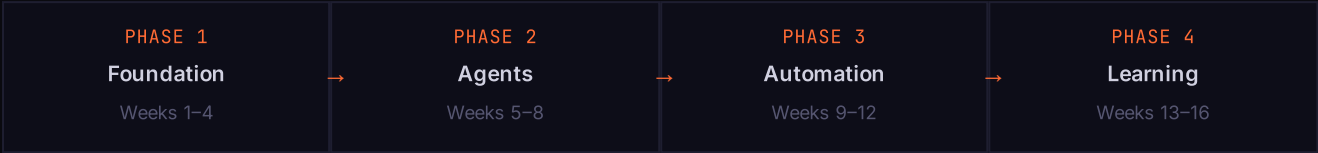
1.1 System Summary

The platform is a **full-stack TypeScript/Python monorepo** that automates the entire job application lifecycle: discovery → evaluation → resume tailoring → cover letter generation → form submission → inbox management → outcome tracking → strategy adjustment. It is grounded in Vikram's demonstrated track record — 92% automation efficiency gains at ATO, \$5M+ portfolio management at ANZ, 38% LLM error-rate reduction — and targets AI Architect, Senior Delivery Lead, and Technical Product Owner roles across ANZ, NAB, Telstra, and government agencies in Australia.

20	4	16	3	0.85
AI AGENTS	BUILD PHASES	WEEKS	JOB BOARDS	AUTO-APPLY THRESHOLD



1.2 Build Order and Dependency Chain



The dependency chain is strict: **Phase 1 must be complete before Phase 2 begins**, because all agent definitions reference the database schema (Prisma models), the queue infrastructure (BullMQ), and the API contract (REST + tRPC). Phase 3 (automation) depends on the working agents from Phase 2. Phase 4 (learning) depends on the application submission data generated in Phase 3.

Table 1.1 Critical path dependencies

Deliverable	Depends On	Blocks
Prisma Schema	Environment bootstrap	All agents, all APIs
BullMQ queues	Redis, Prisma schema	Job discovery, agent dispatch
FastAPI /api/agents/*	LangGraph, BullMQ	Frontend approval UI
pgvector embeddings	PostgreSQL + extension	ATS agent, semantic search
Playwright scraper	Phase 1 complete	Job discovery pipeline
Langfuse project	API key provisioned	All agents (tracing)

1.3 Environment Requirements

Table 1.2 Minimum environment requirements

Component	Minimum Version	Purpose
Node.js	20.x LTS	Next.js 14 frontend, BullMQ workers
pnpm	8.x	Monorepo package manager (workspaces)
Python	3.11+	FastAPI backend, LangGraph agents
Docker	24.x	Local dev containers, CI builds
Docker Compose	2.x	Local orchestration (DB, Redis, workers)
kubectl	1.28+	K8s cluster management
Terraform	1.6+	AWS infrastructure provisioning
PostgreSQL	16 + pgvector	Primary database + embedding store
Redis	7.x	BullMQ queues, session cache
AWS CLI	2.x	EKS, RDS, S3, SQS provisioning

1.4 Repository Structure

The repository uses a **pnpm monorepo** layout. All apps and shared packages live in the same repository to simplify cross-package TypeScript type sharing, atomic commits, and coordinated CI pipelines.

```

/platform
├─ apps/
│  └─ web/                                # Next.js 14 App Router frontend
│     └─ app/                             # Route segments (layout, pages, APIs)
│        └─ components/                   # Shared UI components (dark theme)
│           └─ lib/                       # tRPC client, auth helpers
│              └─ public/                 # Static assets
│                 └─ api/                 # FastAPI backend (Python)
│                    └─ routers/           # Endpoint route handlers
│                       └─ agents/         # LangGraph agent modules
│                          └─ workers/     # BullMQ worker processes
│                             └─ models/    # SQLAlchemy / Pydantic models
│                                └─ services/ # Business logic layer
├─ packages/
│  └─ agents/                             # LangGraph graph definitions (shared)
│     └─ graphs/                         # One file per agent workflow
│        └─ tools/                       # Tool function definitions
│           └─ prompts/                  # System prompt templates
│              └─ schemas/               # Input/Output Zod/Pydantic schemas
│                 └─ db/                 # Prisma schema + migrations
│                    └─ schema.prisma    # Single source-of-truth schema
│                       └─ migrations/   # Auto-generated migration history
│                          └─ seed.ts     # Seed data for local dev
│                             └─ shared/  # Cross-app TypeScript types
│                                └─ types/ # Domain types (Job, Application, etc.)
│                                   └─ utils/ # Shared utilities (scoring, parsing)
│                                      └─ constants/ # Platform-wide constants / thresholds
│                                         └─ queue/   # BullMQ job definitions
│                                            └─ jobs/   # Job type definitions + payloads
│                                               └─ processors/ # Queue processor factories
│                                                  └─ schedulers/ # Cron-based scheduled jobs
├─ infra/
│  └─ terraform/                         # Terraform modules (EKS, RDS, S3)
│     └─ k8s/                           # Kubernetes manifests
│        └─ docker/                     # Production Dockerfiles
├─ .github/workflows/                   # CI/CD GitHub Actions
├─ docker-compose.yml                  # Local development stack
├─ pnpm-workspace.yaml                  # Workspace definition
└─ turbo.json                           # Turborepo build pipeline

```

AI CODING AGENT NOTE

When initializing this repository, run `pnpm install` from the root. All package resolution is handled by pnpm workspaces. To add a dependency to a specific app: `pnpm --filter @platform/api add langchain`. To run all dev servers: `pnpm dev` (Turborepo orchestrates the startup order).

SECTION 02 • PHASE 1 • WEEKS 1-4

Foundation

Project scaffolding, database schema, API contracts, and authentication

2.1 Project Scaffolding

Run the following commands in sequence to initialize the monorepo from scratch. Each command is idempotent — safe to re-run if a step fails mid-way.

```
# 1. Bootstrap monorepo
mkdir platform && cd platform
pnpm init
echo "packages:\n  - 'apps/*'\n  - 'packages/*'" > pnpm-workspace.yaml

# 2. Initialize turbo
pnpm add -Dw turbo typescript @types/node
npx turbo init

# 3. Create apps
pnpm create next-app apps/web --typescript --tailwind --app --src-dir
mkdir -p apps/api && cd apps/api
python3 -m venv .venv && source .venv/bin/activate
pip install fastapi uvicorn langgraph langchain openai prisma

# 4. Create packages
for pkg in agents db shared queue; do
  mkdir -p packages/$pkg && cd packages/$pkg
  pnpm init && cd ../..
done

# 5. Start local dev stack
docker compose up -d # Starts PostgreSQL, Redis, pgAdmin
```

docker-compose.yml (local development)

```
version: '3.9'
services:
  postgres:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: platform
      POSTGRES_PASSWORD: platform_dev
      POSTGRES_DB: career_platform
    ports: ["5432:5432"]
    volumes: [postgres_data:/var/lib/postgresql/data]

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
    command: redis-server --maxmemory 512mb --maxmemory-policy allkeys-lru

  worker:
    build: ./apps/api
    command: python -m workers.main
    depends_on: [postgres, redis]
    environment:
      DATABASE_URL: postgresql://platform:platform_dev@postgres/career_platform
      REDIS_URL: redis://redis:6379

volumes:
  postgres_data:
```

2.2 Database Schema (Prisma)

The complete Prisma schema is the authoritative data contract for the platform. Save this to `packages/db/schema.prisma`. Run `pnpm prisma migrate dev --name init` to generate the first migration.

```

// packages/db/schema.prisma
generator client {
  provider      = "prisma-client-js"
  previewFeatures = ["postgresqlExtensions"]
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
  extensions = [pgvector(map: "vector")]
}

// — ENUMS —
enum ApplicationStatus {
  DISCOVERED EVALUATED TAILORING_QUEUED TAILORING_COMPLETE
  SUBMITTED ACKNOWLEDGED INTERVIEW_SCHEDULED OFFER REJECTED
  WITHDRAWN ARCHIVED
}

enum ConfidenceLevel { AUTO_APPLY NEEDS_APPROVAL SKIP }
enum JobSource        { LINKEDIN SEEK INDEED GLASSDOOR RSS DIRECT }
enum AgentStatus       { PENDING RUNNING WAITING_APPROVAL COMPLETE FAILED }
enum InterviewType     { PHONE TECHNICAL BEHAVIOURAL PANEL FINAL }
enum OAuthProvider     { GOOGLE GITHUB LINKEDIN }

// — USER & PROFILE —
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  passwordHash String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  profile     Profile?
  accounts    OAuthAccount[]
  sessions    Session[]
  auditLogs   AuditTrail[]
  @@index([email])
}

model OAuthAccount {
  id          String    @id @default(cuid())
  userId      String
  provider    OAuthProvider
  providerAccountId String
  accessToken  String?  @db.Text
  refreshToken String?  @db.Text
  expiresAt   DateTime?
  user        User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@unique([provider, providerAccountId])
}

model Session {
  id          String    @id @default(cuid())
  userId      String
  token       String    @unique
  expiresAt   DateTime
  createdAt   DateTime  @default(now())
  user        User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  @@index([token])
}

model Profile {
  id          String    @id @default(cuid())
  userId      String    @unique

```

POST-MIGRATION STEP

After running migrations, enable pgvector: `psql -U platform career_platform -c "CREATE EXTENSION IF NOT EXISTS vector;"`. Then add the IVFFlat index for fast similarity search: `CREATE INDEX ON "Embedding" USING ivfflat (vector vector_cosine_ops) WITH (lists = 100);`

2.3 API Contract Specification

All endpoints are prefixed `/api`. REST endpoints are handled by FastAPI. tRPC routes (used in the Next.js frontend) are defined in `apps/web/lib/trpc.ts` and proxy through to the FastAPI backend. All authenticated routes require a valid JWT in the `Authorization: Bearer <token>` header.

AUTH

POST	/api/auth/register	Register new user — body: <code>{ email, password, fullName }</code>	public
POST	/api/auth/login	Login → returns <code>{ accessToken, refreshToken, user }</code>	public
GET	/api/auth/me	Get current user profile and session info	JWT
POST	/api/auth/oauth/:provider	Initiate OAuth flow (google github linkedin)	public
POST	/api/auth/refresh	Refresh access token — body: <code>{ refreshToken }</code>	public

PROFILE

GET	/api/profile	Get full profile including skills, achievements, portfolio	JWT
PUT	/api/profile	Update profile — partial update supported (PATCH semantics)	JWT
POST	/api/profile/resume/upload	Upload base resume PDF — multipart/form-data — triggers parse agent	JWT · 10MB
GET	/api/profile/portfolio/sync	Trigger portfolio sync from <code>forgotten-mistory.web.app</code>	JWT

JOBS

GET	/api/jobs	List jobs — query: <code>source, minScore, status, page, limit, sort</code>	JWT · 60/min
GET	/api/jobs/:id	Get job detail including match/ATS scores and embedding metadata	JWT
POST	/api/jobs/:id/evaluate	Manually trigger evaluation agent for a specific job	JWT · 10/min
POST	/api/jobs/:id/apply	Approve and submit application — body: <code>{ resumeId, coverLetterId }</code>	JWT · 5/min

APPLICATIONS

GET	/api/applications	List applications — query: <code>status, dateFrom, dateTo, page, limit</code>	JWT
GET	/api/applications/:id	Get application with status history, agent actions, and documents	JWT
PUT	/api/applications/:id/status	Manually update status — body: <code>{ status, reason }</code>	JWT
GET	/api/applications/:id/timeline	Get full event timeline (status changes, agent actions, emails)	JWT

AGENTS

POST	/api/agents/tailor-resume	Queue resume tailoring — body: { resumeId, jobId } → returns { taskId }	JWT • 20/hr
POST	/api/agents/generate-cover-letter	Generate cover letter — body: { applicationId, tone?, length? }	JWT • 20/hr
POST	/api/agents/answer-question	Answer application form question — body: { question, context, jobId }	JWT • 50/hr
GET	/api/agents/status/:taskId	Poll agent task status — returns { status, progress, result? }	JWT
POST	/api/agents/approve/:taskId	Human approval for tasks at WAITING_APPROVAL checkpoint	JWT

ANALYTICS

GET	/api/analytics/funnel	Application funnel — counts per status stage, conversion rates	JWT
GET	/api/analytics/performance	Response rate, interview rate, offer rate by job source and role type	JWT
GET	/api/analytics/agent-metrics	Agent latency, error rate, token usage, cost per action	JWT

Key TypeScript Interfaces

```
// Shared response envelope — all API responses
interface ApiResponse<T> {
  success: boolean
  data: T
  error?: { code: string; message: string }
  meta?: { page: number; total: number; limit: number }
}

// POST /api/agents/tailor-resume
interface TailorResumeRequest { resumeId: string; jobId: string }
interface TailorResumeResponse { taskId: string; estimatedSeconds: number }

// GET /api/agents/status/:taskId
interface AgentStatusResponse {
  taskId: string
  status: 'PENDING' | 'RUNNING' | 'WAITING_APPROVAL' | 'COMPLETE' | 'FAILED'
  progress: number // 0-100
  result?: ResumeVersion | CoverLetterVersion
  checkpoint?: { message: string; approvalRequired: boolean }
  error?: string
}

// GET /api/analytics/funnel
interface FunnelStage {
  status: ApplicationStatus
  count: number
  conversionRate: number // % from previous stage
}
type FunnelResponse = FunnelStage[]
```

2.4 Authentication Setup

2.4.1 NextAuth.js Configuration

```
// apps/web/app/api/auth/[...nextauth]/route.ts
import NextAuth from 'next-auth'
import GoogleProvider from 'next-auth/providers/google'
import GitHubProvider from 'next-auth/providers/github'
import LinkedInProvider from 'next-auth/providers/linkedin'
import CredentialsProvider from 'next-auth/providers/credentials'

export const authOptions = {
  providers: [
    GoogleProvider({ clientId: process.env.GOOGLE_CLIENT_ID!, clientSecret:
process.env.GOOGLE_CLIENT_SECRET! }),
    GitHubProvider({ clientId: process.env.GITHUB_CLIENT_ID!, clientSecret:
process.env.GITHUB_CLIENT_SECRET! }),
    LinkedInProvider({ clientId: process.env.LINKEDIN_CLIENT_ID!, clientSecret:
process.env.LINKEDIN_CLIENT_SECRET! }),
    CredentialsProvider({
      name: 'credentials',
      credentials: { email: { type: 'email' }, password: { type: 'password' } },
      async authorize(credentials) {
        const res = await fetch(`${process.env.API_URL}/api/auth/login`, {
          method: 'POST', body: JSON.stringify(credentials),
          headers: { 'Content-Type': 'application/json' }
        })
        const data = await res.json()
        return res.ok ? data.user : null
      }
    })
  ],
  session: { strategy: 'jwt', maxAge: 30 * 24 * 60 * 60 },
  callbacks: {
    async jwt({ token, user }) {
      if (user) { token.userId = user.id; token.role = user.role }
      return token
    },
    async session({ session, token }) {
      session.user.id = token.userId as string
      return session
    }
  }
}
```

2.4.2 JWT Structure

```
// JWT payload — issued by FastAPI, verified by Next.js middleware
{
  "sub": "cuid_user_id",
  "email": "vikram@example.com",
  "role": "user",
  "iat": 17200000000,
  "exp": 1720086400,    // 24h access token
  "jti": "unique_token_id" // for revocation
}

// Environment variables required
NEXTAUTH_SECRET="min-32-char-secret"
NEXTAUTH_URL="http://localhost:3000"
JWT_SECRET="same-secret-used-in-fastapi"
ACCESS_TOKEN_EXPIRE_MINUTES=1440
REFRESH_TOKEN_EXPIRE_DAYS=30
```

LINKEDIN OAUTH NOTE

LinkedIn OAuth v2 requires the `r_liteprofile` and `r_emailaddress` scopes. The callback URL must be registered in the LinkedIn Developer Portal as `{NEXTAUTH_URL}/api/auth/callback/linkedin`. LinkedIn profile sync for job discovery requires a separate `w_member_social` scope and must be requested explicitly in the OAuth consent flow.

SECTION 03 • PHASE 2 • WEEKS 5-8

Agent Implementation

LangGraph framework, 20 agent specifications, and orchestration workflows

3.1 Agent Framework Setup

All agents are built on **LangGraph** — a graph-based orchestration framework that enables stateful, multi-step agent workflows with first-class support for human-in-the-loop checkpoints. Each agent is a directed graph: nodes are Python functions, edges are conditional routing logic.

3.1.1 Core State Schema

```
# packages/agents/schemas/base.py
from typing import TypedDict, Optional, List, Annotated
from langgraph.graph import add_messages

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]
    job_id: str
    application_id: Optional[str]
    resume_id: Optional[str]
    context: dict # Vikram's profile, skills, achievements
    tool_results: List[dict]
    current_step: str
    approval_needed: bool
    error: Optional[str]
    trace_id: Optional[str] # Langfuse trace ID
```

3.1.2 Graph Boilerplate

```
# packages/agents/graphs/base_graph.py
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.postgres import PostgresSaver
from langfuse.langchain import CallbackHandler
import os

def build_graph(state_class, nodes: dict, edges: list, checkpoints=True):
    graph = StateGraph(state_class)
    for name, fn in nodes.items():
        graph.add_node(name, fn)
    for src, dst, condition in edges:
        if condition:
            graph.add_conditional_edges(src, condition)
        else:
            graph.add_edge(src, dst)
    checkpointer = PostgresSaver.from_conn_string(
        os.environ["DATABASE_URL"]
    ) if checkpoints else None
    return graph.compile(checkpointer=checkpointer, interrupt_before=["human_review"])

# Langfuse callback - attach to every LLM call
langfuse_handler = CallbackHandler(
    public_key=os.environ["LANGFUSE_PUBLIC_KEY"],
    secret_key=os.environ["LANGFUSE_SECRET_KEY"],
    host=os.environ.get("LANGFUSE_HOST", "https://cloud.langfuse.com")
)
```

3.1.3 Tool Definition Pattern

```
# packages/agents/tools/job_tools.py
from langchain_core.tools import tool
from pydantic import BaseModel, Field

class SearchJobsInput(BaseModel):
    query: str = Field(description="Job title or keyword")
    location: str = Field(default="Melbourne, VIC")
    source: str = Field(default="linkedin")
    max_results: int = Field(default=20)

@tool
async def search_jobs(input: SearchJobsInput) → list[dict]:
    """Search job boards for matching positions."""
    # Implementation: Playwright scraper → returns list of job dicts
    from workers.scrapers import get_scraper
    scraper = get_scraper(input.source)
    return await scraper.search(input.query, input.location, input.max_results)
```

3.1.4 Error Handling Pattern

```
# Standard error handling for all agent nodes
import functools
from tenacity import retry, stop_after_attempt, wait_exponential

def agent_node(name: str):
    def decorator(fn):
        @retry(stop=stop_after_attempt(3), wait=wait_exponential(min=1, max=10))
        @functools.wraps(fn)
        async def wrapper(state: AgentState) → AgentState:
            try:
                result = await fn(state)
                await log_agent_action(name, state["trace_id"], result)
                return result
            except Exception as e:
                await log_error(name, state, str(e))
                return {**state, "error": str(e), "current_step": "failed"}
        return wrapper
    return decorator
```

3.2 Individual Agent Specifications (20 Agents)

Each specification below defines the exact system prompt, input/output schema, tool list, and example invocation for every agent in the platform. All agents are housed in `packages/agents/graphs/` with corresponding prompt files in `packages/agents/prompts/`.

3.2.1 Tier 1 — Discovery & Evaluation (Agents 1–4)

1. Job Discovery Agent

discovery_agent

Scans LinkedIn, Seek, and Indeed every 30 minutes for roles matching Vikram's target profile. Scores raw listings and forwards above-threshold jobs to the Evaluation Agent.

Model: gpt-4o-mini Tools: search_linkedin, search_seek, search_indeed, filter_by_criteria Trigger: CRON every 30min

```
SYSTEM_PROMPT = """You are the Job Discovery Agent for Vikram Deshpande, a Melbourne-based
Technical Delivery Lead and AI Solutions Architect (15+ years experience).
```

```
TARGET ROLES: AI Solutions Architect | Senior Delivery Lead | Technical Product Owner
| Scrum Master | Platform Engineering Lead | AI/ML Program Manager
```

```
TARGET COMPANIES: ANZ, NAB, Telstra, AT0, Westpac, Commonwealth Bank, Atlassian, Canva,
KPMG, Deloitte, Accenture, AWS, Microsoft, Google
```

```
LOCATION: Melbourne, VIC (remote/hybrid acceptable)
```

```
SALARY RANGE: AUD 180,000-230,000
```

```
FILTERING RULES:
```

1. Match score = weighted_sum(title_match * 0.35, skills_overlap * 0.30, company_target * 0.20, location_match * 0.15)
2. Forward to Evaluation Agent if match_score ≥ 0.75
3. Flag NEEDS_APPROVAL if match_score 0.65-0.74
4. SKIP if match_score < 0.65 or if role is purely individual-contributor (no leadership)
5. SKIP duplicate postings (same role within 7 days at same company)

```
OUTPUT: Return structured job objects with source, url, match_score, reasoning."""
```

2. Job Evaluation Agent

evaluation_agent

Deep-dives into a specific job posting to produce a comprehensive fit analysis, ATS keyword extraction, and application recommendation. Reads the full JD and cross-references against Vikram's profile.

Model: gpt-4o Tools: get_job_details, get_profile, compute_ats_score, get_similar_roles

```
SYSTEM_PROMPT = """You are the Job Evaluation Agent. Given a job description and Vikram
Deshpande's profile, produce a detailed fit analysis.
```

```
VIKRAM'S KEY DIFFERENTIATORS:
```

- 92% automation efficiency gain at ATO (COBOL/mainframe test harness)
- \$5M+ portfolio management at ANZ with 100% compliance
- Real-time telemetry (10k+ concurrent devices, P95 latency < 200ms)
- LangChain/Langfuse LLM pipeline with 38% error reduction
- 15+ years spanning government, finance, and telco
- Multi-Agent Orchestrator (JARVIS) – GitHub: @Victordtesla24

```
OUTPUT FORMAT:
```

```
{
  "overall_score": 0.0-1.0,
  "ats_score": 0-100,
  "key_matches": ["..."],
  "gaps": ["..."],
  "recommended_emphasis": ["..."], // which achievements to highlight
  "confidence_level": "AUTO_APPLY | NEEDS_APPROVAL | SKIP",
  "reasoning": "..."
}
```

3. ATS Optimization Agent

ats_agent

Runs the ATS scoring algorithm against a resume version and job description. Produces keyword injection recommendations and validates the final resume scores above threshold.

Model: gpt-4o Tools: extract_keywords, compare_embeddings, get_resume_text

```
# ATS scoring algorithm
def calculate_ats_score(resume_text: str, job_description: str) → dict:
    # 1. Hard keyword matching (exact + stemmed)
    jd_keywords = extract_keywords(job_description, top_n=50)
    resume_terms = extract_keywords(resume_text, top_n=100)
    keyword_score = jaccard_similarity(jd_keywords, resume_terms) * 100

    # 2. Semantic similarity (pgvector cosine distance)
    jd_vec = embed_text(job_description)
    resume_vec = embed_text(resume_text)
    semantic_score = (1 - cosine_distance(jd_vec, resume_vec)) * 100

    # 3. Experience level match
    exp_score = evaluate_experience_match(
        resume_years=15, required_years=parse_years_required(job_description)
    ) * 100

    # Weighted final score
    final = (0.45 * keyword_score + 0.35 * semantic_score + 0.20 * exp_score)
    return { "score": round(final, 1), "keyword": keyword_score,
            "semantic": semantic_score, "experience": exp_score,
            "missing_keywords": list(jd_keywords - resume_terms) }
```

4. Company Research Agent

company_agent

Researches a target company before application — culture, recent news, leadership, current tech stack, open headcount signals — to inform cover letter personalization.

Model: gpt-4o Tools: web_search, scrape_linkedin_company, get_glassdoor_reviews

```
SYSTEM_PROMPT = """Research [company_name] for a job application by Vikram Deshpande.
Focus on:
1. Recent AI/digital transformation initiatives (last 12 months)
2. Engineering culture and team structure
3. Key pain points this role likely addresses
4. Named executives and their LinkedIn presence
5. Tech stack signals from job ads and engineering blogs

Format as structured JSON for use by the Cover Letter Agent."""
```

3.2.2 Tier 2 — Document Generation (Agents 5–9)

5. Resume Tailoring Agent

resume_tailor_agent

The most critical agent. Tailors Vikram's base resume for a specific job while preserving the exact two-column, coral/orange-accented visual layout. Pipeline: parse → enrich → map → rewrite → validate → export.

Model: gpt-4o Tools: parse_resume_json, get_job_evaluation, reorder_bullets, validate_ats_score, export_pdf

Checkpoint: human_review

```
SYSTEM_PROMPT = """You are the Resume Tailoring Agent for Vikram Deshpande.
```

```
CRITICAL FORMATTING RULES – NEVER VIOLATE:
```

1. PRESERVE the two-column asymmetric layout (left sidebar 30%, main 70%)
2. PRESERVE the coral/orange (#FF6B35) title box and section icons
3. PRESERVE the Inter/Montserrat font hierarchy
4. NEVER add new sections – only reorder, rephrase, and emphasize existing content
5. MAX 15 content changes per tailoring pass (bullets reordered/reworded)
6. NEVER fabricate metrics or achievements – only use what is in source_resume

```
TAILORING STRATEGY:
```

1. Move most-relevant experience to top of each section
2. Mirror exact keywords from job description in first bullet of each role
3. Quantify every achievement – use Vikram's actual metrics:
 - AT0: 92% effort reduction, 3hrs → 15min, 95%+ completion rate
 - ANZ: 30% delivery efficiency, 15% cost reduction, 10k+ devices, P95<200ms
 - AI: 38% error reduction, \$5M+ portfolio, 40+ resources, 100% compliance
4. Ensure ATS score ≥ 70 before approving output
5. If ATS score < 70 after 2 rewrites, flag NEEDS_HUMAN_REVIEW

```
PIPELINE: parse_json → keyword_inject → reorder_bullets → validate_ats → export_pdf
```

```
CHECKPOINT: Always interrupt for human_review before final PDF export."""
```

6. Cover Letter Agent

cover_letter_agent

Generates a personalised, compelling cover letter in Vikram's voice. Integrates company research, role context, and specific achievement evidence from the resume.

Model: gpt-4o Tools: get_company_research, get_job_evaluation, get_achievements_by_relevance

```
SYSTEM_PROMPT = """Write a cover letter for Vikram Deshpande applying to [job_title] at [company_name]. Style: confident, data-driven, forward-looking. Tone: senior executive but personable. NOT generic – every paragraph must reference specifics.
```

```
STRUCTURE (max 400 words, 4 paragraphs):
```

1. Hook: Name the specific program/initiative at [company] that excites Vikram. Why now.
2. Evidence: 2-3 most relevant quantified achievements (use actual numbers from profile)
3. Insight: One specific technical or strategic insight about [company]'s challenge
4. CTA: Clear next step, mention portfolio at <https://forgotten-mistory.web.app>

```
VOICE RULES:
```

- First person, active verbs
- No "I am excited to apply" openers
- Lead with impact, end with specificity
- Reference Vikram's GitHub @Victordtesla24 for technical roles

```
OUTPUT: Plain text, max 400 words. No HTML. No markdown formatting."""
```

7. Portfolio Sync Agent

portfolio_agent

Crawls forgotten-mistory.web.app and GitHub @Victordtesla24 to extract project updates, new metrics, and technology additions for use by the Resume and Cover Letter agents.

Model: gpt-4o-mini Tools: scrape_portfolio_url, fetch_github_repos, update_portfolio_db Trigger: CRON daily

```
SYSTEM_PROMPT = """Extract structured project data from Vikram Deshpande's portfolio at https://forgotten-mistory.web.app and GitHub @Victordtesla24.
```

```
For each project, extract:
```

- name, description, tech_stack[], metrics[], github_url, demo_url
- evidence_links[] (YouTube demos, documentation URLs)

```
Known projects to update: JARVIS (telemetry/Jira), ATO Evidence Harness (COBOL), Vedic Astronomy Platform, Multi-Agent Orchestrator, AI Image Enhancer. Flag any new projects not in the database for manual review."""
```

8. Question Answering Agent

qa_agent

Answers free-text application questions (motivation questions, situational/behavioural prompts, salary expectations, availability) drawing exclusively from Vikram's verified profile data.

Model: gpt-4o Tools: get_profile_context, retrieve_similar_answers, get_achievements

```
SYSTEM_PROMPT = """Answer the application question below as Vikram Deshpande.
```

```
CONTEXT RULES:
```

- Use ONLY verified data from Vikram's profile – never fabricate
- For behavioural questions: use STAR format (Situation, Task, Action, Result)
- For technical questions: reference specific projects from GitHub @Victordtesla24
- For salary questions: respond "AUD 180,000-230,000 depending on total package"
- For availability: respond "Available immediately / 2 weeks notice"
- Max 250 words unless the question explicitly requires more
- Match the tone of the question (formal, casual, technical)"""

9. Document Quality Agent

quality_agent

Final quality gate before submission. Checks resume and cover letter for grammar, ATS score, formatting integrity, factual accuracy (no hallucinated metrics), and completeness.

Model: gpt-4o Tools: grammar_check, validate_metrics_against_profile, compute_ats_score

```
SYSTEM_PROMPT = """Perform quality assurance on the resume/cover letter before submission.
```

```
MANDATORY CHECKS (ALL must pass):
```

1. ATS score $\geq$ 70 (reject if lower – return to Resume Tailor Agent)
2. Zero hallucinated metrics (validate every number against source profile)
3. No formatting anomalies (2-column layout intact, fonts consistent)
4. Cover letter < 400 words, resume $\leq$ 2 pages
5. Contact details present: Melbourne location, LinkedIn, GitHub @Victordtesla24
6. No generic phrases: "team player", "hard worker", "results-driven"

```
OUTPUT: { "passed": bool, "issues": [], "ats_score": float, "word_count": int }
If passed=false, list specific issues with suggested fixes."""
```

3.2.3 Tier 3 — Submission & Communication (Agents 10–14)

10. Form Detection Agent

form_agent

Uses Playwright to analyze an application form structure — field labels, types, required/optional status — and generates a field mapping JSON for the Submission Agent.

Model: gpt-4o-mini Tools: playwright_get_form, analyze_dom_structure, map_fields_to_profile

```
SYSTEM_PROMPT = """Analyze the application form DOM structure and map each field to
Vikram's profile data.

FIELD MAPPING PRIORITY:
1. Identify all required fields (look for *, 'required', aria-required=true)
2. Map standard fields: name, email, phone, location, resume upload, cover letter
3. Detect custom questions — pass each to the QA Agent for answering
4. Flag file upload fields — map to pre-generated resume/cover letter PDFs
5. Detect work authorization questions: answer "Australian Citizen/PR"

OUTPUT: { "fields": [{ "selector": "", "type": "", "mapped_value": "", "confidence": 0.0 }],
          "custom_questions": [], "file_uploads": [] }"""
```

11. Application Submission Agent

submission_agent

Executes the automated form submission via Playwright. Fills fields, uploads files, clicks submit, and captures screenshot evidence. Verifies successful submission via confirmation page detection.

Model: gpt-4o Tools: playwright_fill_form, playwright_upload_file, playwright_screenshot, verify_confirmation

Checkpoint: human_review (confidence < 0.85)

```
SYSTEM_PROMPT = """Execute the job application form submission for Vikram Deshpande.

SUBMISSION RULES:
1. STOP and await human approval if confidence < 0.85 on any field mapping
2. Take pre-fill screenshot (evidence of form state before submission)
3. Fill fields in order: identity → contact → work history → documents → questions
4. For file uploads: use pre-generated tailored resume PDF path
5. Verify submission: look for confirmation text, order ID, or redirect to success page
6. Take post-submit screenshot as evidence
7. If submission fails: do NOT retry automatically — flag for human review
8. Rate limit: max 10 applications per day, min 2 minute gap between submissions

EVIDENCE: Store screenshot_url, confirmation_text, submitted_at in database."""
```

12. Email Classification Agent

email_agent

Monitors Vikram's inbox for application-related emails. Classifies each email (confirmation, rejection, interview request, recruiter outreach) and routes to the appropriate response agent.

Model: gpt-4o-mini Tools: gmail_list_unread, classify_email, extract_calendar_invite, update_application_status

```
SYSTEM_PROMPT = """Classify incoming emails related to job applications for Vikram Deshpande.
```

```
CLASSIFICATION RULES:
```

- APPLICATION_CONFIRMATION: automated receipt, "we have received your application"
- REJECTION: "not moving forward", "other candidates", "unsuccessful"
- INTERVIEW_REQUEST: scheduling language, Calendly links, proposed times
- RECRUITER_OUTREACH: unsolicited recruiter contact with new opportunity
- INFORMATION_REQUEST: additional documents or information needed
- OFFER: compensation details, start date, contract attached
- UNKNOWN: forward to human review

```
OUTPUT: { "classification": "", "company": "", "role": "", "action_required": bool,
           "suggested_response": "", "application_id": "" }"""
```

13. Email Response Agent

email_response_agent

Drafts responses to recruiter emails and interview requests in Vikram's voice. All responses require human approval before sending.

Model: gpt-4o Checkpoint: human_review (always)

```
SYSTEM_PROMPT = """Draft an email response as Vikram Deshpande to [email_classification].
```

```
TONE GUIDELINES:
```

- Recruiter outreach: warm, direct, ask for specifics (company, salary, role)
- Interview request: enthusiastic, confirm details, suggest times if asked
- Information request: prompt, thorough, professional
- NEVER auto-respond to offers without explicit human approval

```
SIGN-OFF: "Kind regards, Vikram Deshpande | LinkedIn: vikramdeshpande.io"
```

```
ALWAYS flag as NEEDS_HUMAN_APPROVAL before sending."""
```

14. Calendar Agent

calendar_agent

Extracts interview times from emails and calendar invites. Creates Interview records in the database and triggers the Interview Prep Agent. Detects conflicts with existing calendar events.

Model: gpt-4o-mini Tools: extract_datetime_nlp, google_calendar_check, create_interview_record

```
SYSTEM_PROMPT = """Extract interview scheduling details and create a calendar entry.
```

```
EXTRACTION RULES:
```

1. Parse natural language dates: "next Tuesday at 2pm AEST" → ISO 8601
2. Detect interview type: phone, video (Zoom/Teams/Meet link present), on-site (address)
3. Check Google Calendar for conflicts on that day
4. Create Interview record: { scheduled_at, type, location/url, company, role }
5. Auto-trigger Interview Prep Agent 24h before scheduled time"""

3.2.4 Tier 4 — Interview & Learning (Agents 15–20)

15. Interview Prep Agent

interview_prep_agent

Generates 20–30 role-specific interview questions with structured STAR answers drawn from Vikram's profile. Covers technical, behavioural, and situational questions for the specific company and role.

Model: gpt-4o Tools: get_job_evaluation, get_company_research, get_achievements, get_past_prep_sessions

```
SYSTEM_PROMPT = """Generate interview preparation materials for Vikram Deshpande
interviewing for [job_title] at [company_name].
```

```
QUESTION CATEGORIES (min 5 per category):
```

1. Behavioural: STAR-format answers using specific ANZ/ATO/NAB achievements
2. Technical: AI/ML architecture, LangGraph, Langfuse, Kubernetes, COBOL legacy
3. Leadership: team of 40 at ANZ, \$5M portfolio, PI planning at ATO
4. Situational: "How would you handle..." scenarios specific to [company]'s context
5. Questions to ask: 3 insightful questions that demonstrate research into [company]

```
ANSWER FORMAT per question:
```

```
{ "question": "", "category": "", "answer": { "S": "", "T": "", "A": "", "R": "" },
  "key_metrics": [], "follow_up_risk": "" }
```

16. Outcome Tracking Agent

outcome_agent

Records application outcomes (response received, interview rate, offer rate) and links outcomes to the specific resume version, cover letter, and agent parameters used. Powers the learning pipeline.

Model: gpt-4o-mini Tools: update_application_status, record_learning_outcome, compute_feature_vector

```
SYSTEM_PROMPT = """Record and classify the outcome of job application [application_id].
```

```
FEATURE EXTRACTION for learning:
```

- ats_score, match_score, cover_letter_length, tailoring_changes_count
- company_size, industry, job_source, days_since_posting
- response_time_days (null if no response after 21 days = implicit rejection)
- interview_achieved (bool), offer_achieved (bool)

```
Store as LearningOutcome record for use by Strategy Adjustment Agent."""
```

17. Strategy Adjustment Agent

strategy_agent

Analyzes accumulated outcome data to identify which parameters are driving interview rates. Adjusts thresholds, prompts, and targeting parameters with statistical confidence gates.

Model: gpt-4o Trigger: CRON weekly + after 10 new outcomes

```
SYSTEM_PROMPT = """Analyze application outcomes and recommend parameter adjustments.
```

```
ANALYSIS REQUIREMENTS:
```

- Minimum 20 applications before making threshold adjustments
- Statistical significance: $p < 0.05$ using two-sample proportion test
- Compare: high_ats (≥ 80) vs low_ats (70-79) interview rates
- Compare: cover_letter_short ($< 300w$) vs long ($> 300w$) response rates
- Segment by: job_source, company_size, role_type

```
ADJUSTMENT LIMITS (never exceed):
```

- min_match_score: floor 0.65, ceiling 0.90
- auto_apply_confidence: floor 0.80, ceiling 0.95
- min_ats_score: floor 65, ceiling 85

```
OUTPUT: [{ "parameter": "", "current_value": "", "recommended_value": "",
            "evidence": "", "expected_improvement": "" }]
```

18. A/B Testing Agent

ab_testing_agent

Manages controlled experiments on resume variants and cover letter styles. Assigns applications to variant A or B, tracks outcomes separately, and reports statistical results.

Model: gpt-4o-mini Tools: assign_variant, track_experiment_outcome, compute_significance

```
SYSTEM_PROMPT = """Manage A/B experiments on application materials.

CURRENT EXPERIMENTS:
- EXP-001: Cover letter opening (hook vs achievement-first) – target n=30
- EXP-002: Resume summary inclusion vs exclusion – target n=40
- EXP-003: Skills section position (sidebar vs body) – target n=50

ASSIGNMENT: Use consistent hashing on application_id for deterministic assignment.
STOPPING: Stop experiment when target n reached or p < 0.05, whichever comes first.
REPORT: Weekly summary with confidence intervals and recommendation."""
```

19. Network Intelligence Agent

network_agent

Identifies first- and second-degree LinkedIn connections at target companies. Suggests warm outreach messages and tracks referral potential before or during application processes.

Model: gpt-4o Tools: search_linkedin_connections, get_company_employees, draft_connection_message

```
SYSTEM_PROMPT = """Identify network connections at [company_name] for Vikram Deshpande.

SEARCH STRATEGY:
1. First-degree: direct connections at [company] (via LinkedIn API)
2. Second-degree: connections of connections
3. Alumni: Monash University (MCom 2010), University of Melbourne (BEng 2007)
4. Past employers: ANZ, NAB, Microsoft, Telstra, AT0 – look for shared alumni

OUTREACH TEMPLATE: Personal, specific, non-transactional. Reference shared context.
Flag: human_review_required = true for all outreach messages."""
```

20. Dashboard Analytics Agent

analytics_agent

Aggregates platform-wide metrics for the Next.js dashboard. Computes funnel conversion rates, agent performance metrics, source effectiveness, and provides weekly summary digests.

Model: gpt-4o-mini Tools: query_analytics_db, compute_conversion_funnel, generate_weekly_digest

Trigger: CRON daily 06:00 AEST

```
SYSTEM_PROMPT = """Generate analytics summary for Vikram's job search dashboard.
```

```
METRICS TO COMPUTE:
```

- Total discovered → evaluated → applied → responded → interviewed → offered
- Conversion rate at each funnel stage
- Best performing: job_source, company_size, role_type, cover_letter_style
- Agent performance: avg latency per agent, error rate, token cost/application
- Time to response (mean, median, P90) by company and source
- Active applications by status with age indicators (>14 days = stale)

```
OUTPUT: Structured JSON for /api/analytics/* endpoints."""
```

3.3 Orchestration Graph

The Master Orchestration Graph coordinates all 20 agents. It is triggered by the Job Discovery Agent's output and routes jobs through the full application pipeline with conditional branching at each decision point.

```

# packages/agents/graphs/orchestration.py
from langgraph.graph import StateGraph, END
from .agents import *

def build_orchestration_graph():
    graph = StateGraph(AgentState)

    # Nodes
    graph.add_node("evaluate", evaluation_agent)
    graph.add_node("research_company", company_research_agent)
    graph.add_node("tailor_resume", resume_tailoring_agent)
    graph.add_node("generate_cl", cover_letter_agent)
    graph.add_node("quality_check", quality_agent)
    graph.add_node("human_review", human_review_node) # interrupt point
    graph.add_node("detect_form", form_detection_agent)
    graph.add_node("submit", submission_agent)
    graph.add_node("track_outcome", outcome_tracking_agent)

    # Entry point
    graph.set_entry_point("evaluate")

    # Conditional routing after evaluation
    def route_after_evaluation(state):
        level = state["confidence_level"]
        if level == "AUTO_APPLY": return "research_company"
        if level == "NEEDS_APPROVAL": return "human_review"
        return END # SKIP

    graph.add_conditional_edges("evaluate", route_after_evaluation,
        { "research_company": "research_company",
          "human_review": "human_review", END: END })

    # Sequential pipeline: research → tailor → cover letter → quality
    graph.add_edge("research_company", "tailor_resume")
    graph.add_edge("tailor_resume", "generate_cl")
    graph.add_edge("generate_cl", "quality_check")

    # Quality gate routing
    def route_after_quality(state):
        if state["quality_passed"] and state["confidence_level"] == "AUTO_APPLY":
            return "detect_form"
        return "human_review" # anything else needs eyes on it

    graph.add_conditional_edges("quality_check", route_after_quality,
        { "detect_form": "detect_form", "human_review": "human_review" })

    # After human approval, resume at detect_form
    graph.add_edge("human_review", "detect_form")
    graph.add_edge("detect_form", "submit")
    graph.add_edge("submit", "track_outcome")
    graph.add_edge("track_outcome", END)

    return graph.compile(
        checkpoint=PostgresSaver.from_conn_string(os.environ["DATABASE_URL"]),
        interrupt_before=["human_review"]
    )

```

PARALLEL EXECUTION

Company Research and ATS scoring run in parallel using LangGraph's `Send()` API. Both complete before the Resume Tailoring Agent begins. This cuts per-application latency by ~40% for the document preparation phase. Implement with: `graph.add_node("parallel_prep", RunnableParallel({"company": company_research_agent, "ats": ats_agent})).`

SECTION 04 · PHASE 3 · WEEKS 9-12

Automation

Job board integration, application submission, and email handling

4.1 Job Board Integration

Job board integration uses Playwright running in headless Chromium. Each scraper is a class implementing the `JobBoardScraper` interface. Rate limiting, session rotation, and anti-detection are handled by the `ScraperManager`.

4.1.1 LinkedIn Scraping Strategy

```
# apps/api/workers/scrapers/linkedin.py
from playwright.async_api import async_playwright
import asyncio, random

class LinkedInScraper:
    BASE_URL = "https://www.linkedin.com/jobs/search/"
    MIN_DELAY = 3.0 # seconds between requests
    MAX_DELAY = 8.0

    async def search(self, query: str, location: str, max_results: int) → list[dict]:
        async with async_playwright() as p:
            browser = await p.chromium.launch(
                headless=True,
                args=["--disable-blink-features=AutomationControlled",
                    "--user-agent=Mozilla/5.0 ..."]
            )
            context = await browser.new_context(
                viewport={"width": 1366, "height": 768},
                locale="en-AU"
            )
            page = await context.new_page()
            await page.goto(self._build_url(query, location))
            jobs = []
            while len(jobs) < max_results:
                items = await page.query_selector_all(".job-search-card")
                for item in items[:max_results - len(jobs)]:
                    job = await self._parse_card(item)
                    if job: jobs.append(job)
                if not await self._has_next_page(page): break
                await asyncio.sleep(random.uniform(self.MIN_DELAY, self.MAX_DELAY))
                await self._click_next(page)
            await browser.close()
            return jobs
```

4.1.2 Rate Limiting and Anti-Detection

Table 4.1 Scraper safety parameters per source

Job Board	Min Delay	Max Requests/hr	Session Rotation	Approach
LinkedIn	3–8s	30	Every 20 requests	Playwright + cookies
Seek	2–5s	60	Every 50 requests	Direct HTTP (Seek has public API)
Indeed	4–10s	20	Every 15 requests	Playwright + residential proxy
RSS Feeds	60s	60/hr per feed	N/A	aiohttp + feedparser

4.2 Application Submission

```
# apps/api/workers/submission/form_filler.py
class FormFiller:
    async def fill_and_submit(self, page, field_mapping: dict, files: dict) → dict:
        evidence = {}
        # Pre-fill screenshot
        evidence["pre_fill"] = await self.screenshot(page, "pre_fill")

        # Fill text fields
        for field in field_mapping["fields"]:
            if field["type"] == "text":
                await page.fill(field["selector"], field["mapped_value"])
            elif field["type"] == "select":
                await page.select_option(field["selector"], field["mapped_value"])
            elif field["type"] == "file":
                await page.set_input_files(field["selector"], files[field["file_type"]])
            await asyncio.sleep(0.3) # human-like typing pace

        # Submit and verify
        await page.click("[type=submit], button:has-text('Apply')")
        await page.wait_for_load_state("networkidle", timeout=15000)
        evidence["post_submit"] = await self.screenshot(page, "post_submit")
        evidence["confirmed"] = await self.detect_confirmation(page)
        return evidence
```

4.3 Email Integration

```
# apps/api/services/email_service.py - Gmail API integration
from google.oauth2.credentials import Credentials
from googleapiclient.discovery import build

class EmailService:
    SCOPES = ["https://www.googleapis.com/auth/gmail.readonly",
              "https://www.googleapis.com/auth/gmail.send"]

    def get_unread_job_emails(self, max_results: int = 50) → list[dict]:
        service = build("gmail", "v1", credentials=self.creds)
        query = "is:unread (subject:application OR subject:interview OR subject:offer)"
        results = service.users().messages().list(
            userId="me", q=query, maxResults=max_results
        ).execute()
        return [self._parse_message(service, m["id"])
                for m in results.get("messages", [])]
```

SECTION 05 • PHASE 4 • WEEKS 13-16

Learning & Scale

Outcome tracking, strategy adjustment, and analytics implementation

5.1 Learning Pipeline

The learning pipeline closes the feedback loop between application outcomes and agent behaviour. Every submitted application generates a `LearningOutcome` record with a feature vector that the Strategy Adjustment Agent analyzes weekly.

5.1.1 Feature Extraction

```
# packages/agents/services/learning.py
def extract_features(application: JobApplication) → dict:
    return {
        "ats_score": application.resume.versions[-1].ats_score,
        "match_score": application.job.match_score,
        "cl_word_count": application.cover_letter.versions[-1].content.split().__len__(),
        "tailoring_changes": application.resume.versions[-1].tailoring_changes_count,
        "job_source": application.job.source,
        "days_since_posted": (application.applied_at - application.job.posted_at).days,
        "company_size_tier": classify_company_size(application.job.company),
        "role_seniority": extract_seniority(application.job.title),
        "ab_variant": application.ab_variant or "control"
    }
```

5.1.2 Strategy Adjustment Algorithm

```
# packages/agents/services/strategy.py
from scipy import stats
import numpy as np

def evaluate_ats_threshold(outcomes: list[LearningOutcome]) → StrategyRecommendation:
    high_ats = [o for o in outcomes if o.features["ats_score"] ≥ 80]
    low_ats = [o for o in outcomes if 70 ≤ o.features["ats_score"] < 80]

    rate_high = np.mean([o.interview_rate for o in high_ats]) if high_ats else 0
    rate_low = np.mean([o.interview_rate for o in low_ats]) if low_ats else 0

    _, p_value = stats.proportions_ztest(
        [sum(o.interview_rate for o in high_ats), sum(o.interview_rate for o in low_ats)],
        [len(high_ats), len(low_ats)]
    )
    if p_value < 0.05 and rate_high > rate_low * 1.2:
        return StrategyRecommendation(
            parameter="min_ats_score", current=70, recommended=75,
            evidence=f"High-ATS rate={rate_high:.2%} vs Low-ATS={rate_low:.2%}, p={p_value:.4f}"
        )
```

5.2 Analytics Implementation

Analytics data is aggregated by the Analytics Agent and served through FastAPI endpoints. All queries use materialized views refreshed hourly to avoid performance degradation on the main transactional tables.

Table 5.1 Key analytics queries

Metric	SQL Approach	Refresh Rate	Alert Threshold
Funnel conversion	COUNT GROUP BY status	Real-time	Response rate < 10%
Interview rate	% INTERVIEW_SCHEDULED / SUBMITTED	Daily	< 15%
Agent error rate	COUNT FAILED / COUNT total by agent	Hourly	> 5% per agent
Token cost/app	SUM tokens × price from AgentLog	Daily	> AUD 2.00/app
Stale applications	WHERE status=SUBMITTED AND appliedAt < NOW()-21d	Daily	Any stale app

SECTION 06

Configuration Specification

Complete platform configuration file with all threshold and tuning parameters

6. Configuration Specification

Save this file as `config/platform.yaml` at the repository root. Load it in all services via the `ConfigService` singleton which validates all values against defined ranges on startup.

```

platform:
  name: "AI Career Agent"
  version: "1.0.0"
  owner: "Vikram Deshpande"
  timezone: "Australia/Melbourne"
  environment: "production" # development | staging | production

profile:
  full_name: "Vikram Deshpande"
  location: "Melbourne, VIC, Australia"
  portfolio_url: "https://forgotten-mistory.web.app"
  github_handle: "@Victordtesla24"
  linkedin_url: "https://vikramdeshpande.io"
  salary_range:
    min_aud: 180000
    max_aud: 230000

thresholds:
  min_match_score: 0.75 # Forward to evaluation agent
  approval_threshold: 0.65 # Flag NEEDS_APPROVAL (0.65-0.74)
  min_ats_score: 70 # Minimum ATS score before submission
  auto_apply_confidence: 0.85 # Submit without human review
  human_approval_required: 0.70 # Always pause for review below this
  duplicate_window_days: 7 # Skip same role at same company
  stale_application_days: 21 # Mark as implicit rejection

agents:
  max_concurrent: 5 # Max parallel agent tasks
  timeout_seconds: 300 # Per-agent execution timeout
  retry_max: 3 # Max retry attempts on failure
  retry_backoff_base: 2 # Exponential backoff base (seconds)
  checkpoint_ttl_hours: 48 # State checkpoint retention
  default_llm: "gpt-4o"
  fast_llm: "gpt-4o-mini"
  embedding_model: "text-embedding-3-small"
  embedding_dimensions: 1536

job_discovery:
  scan_interval_minutes: 30
  max_results_per_scan: 50
  sources:
    - linkedin
    - seek
    - indeed
  target_roles:
    - "AI Solutions Architect"
    - "Senior Delivery Lead"
    - "Technical Product Owner"
    - "Platform Engineering Lead"
    - "AI/ML Program Manager"
    - "Scrum Master"
  target_companies:
    - ANZ
    - NAB
    - Telstra
    - ATO
    - Westpac
    - "Commonwealth Bank"
    - Atlassian
    - Canva
    - AWS
    - Microsoft

```

SECTION 07

Testing Strategy

Unit, integration, and E2E test patterns plus CI/CD pipeline definition

7. Testing Strategy

7.1 Unit Test Patterns (pytest)

```
# tests/unit/agents/test_ats_agent.py
import pytest
from unittest.mock import AsyncMock, patch
from agents.ats_agent import calculate_ats_score

class TestATSScoring:
    def test_high_overlap_returns_high_score(self):
        resume = "LangChain Langfuse Python Kubernetes Docker TypeScript AI/ML"
        jd      = "LangChain Langfuse Python Kubernetes Docker TypeScript AI/ML"
        result = calculate_ats_score(resume, jd)
        assert result["score"] >= 80

    def test_missing_keywords_are_reported(self):
        resume = "Python Docker TypeScript"
        jd      = "Kubernetes Terraform ArgoCD Python"
        result = calculate_ats_score(resume, jd)
        assert "Kubernetes" in result["missing_keywords"]

    def test_zero_overlap_returns_low_score(self):
        result = calculate_ats_score("Marketing Sales", "Kubernetes Python LangGraph")
        assert result["score"] < 30
```

7.2 Integration Test Scenarios

Table 7.1 Integration test scenarios

Scenario	Test File	Pass Condition
Full discovery → evaluation pipeline	test_discovery_pipeline.py	Job stored in DB with match_score
Resume tailor → ATS score >= 70	test_resume_tailoring.py	ResumeVersion.atsScore >= 70
Cover letter generated < 400 words	test_cover_letter.py	word_count <= 400
Agent checkpoint triggers WAITING_APPROVAL	test_orchestration.py	AgentAction.status == WAITING_APPROVAL
Email classified correctly	test_email_classifier.py	classification matches ground truth set
Form fill on mock HTML page	test_form_filler.py	All required fields filled, screenshot saved

7.3 GitHub Actions CI/CD Pipeline

```
# .github/workflows/ci.yml
name: CI Pipeline
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: pgvector/pgvector:pg16
        env: { POSTGRES_PASSWORD: test, POSTGRES_DB: platform_test }
      redis:
        image: redis:7-alpine
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pnpm install
      - run: pnpm prisma migrate deploy
        env: { DATABASE_URL: postgresql://postgres:test@localhost/platform_test }
      - run: pnpm turbo run lint test build
      - run: pip install -r apps/api/requirements.txt
      - run: pytest apps/api/tests/ -v --cov=apps/api --cov-report=xml
      - uses: codecov/codecov-action@v4

  deploy-staging:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ap-southeast-2
      - run: |
          aws eks update-kubeconfig --name platform-eks --region ap-southeast-2
          kubectl apply -f infra/k8s/staging/
          kubectl rollout status deployment/platform-api -n staging
```

SECTION 08

Deployment Specification

Docker Compose, Kubernetes manifests, Terraform modules, and environment catalog

8. Deployment Specification

8.1 Kubernetes Manifests

```
# infra/k8s/production/deployment-api.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: platform-api
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels: { app: platform-api }
  template:
    metadata:
      labels: { app: platform-api }
    spec:
      containers:
        - name: api
          image: <ACCOUNT>.dkr.ecr.ap-southeast-2.amazonaws.com/platform-api:latest
          ports: [{ containerPort: 8000 }]
          resources:
            requests: { cpu: 500m, memory: 512Mi }
            limits: { cpu: 2000m, memory: 2Gi }
          envFrom:
            - configMapRef: { name: platform-config }
            - secretRef: { name: platform-secrets }
          livenessProbe:
            httpGet: { path: /health, port: 8000 }
            initialDelaySeconds: 30
            periodSeconds: 10
          readinessProbe:
            httpGet: { path: /ready, port: 8000 }
            initialDelaySeconds: 10
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: platform-api-svc
  namespace: production
spec:
  selector: { app: platform-api }
  ports: [{ port: 80, targetPort: 8000 }]
  type: ClusterIP
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: platform-api-hpa
  namespace: production
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: platform-api }
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource: { name: cpu, target: { type: Utilization, averageUtilization: 70 } }
```

8.2 Terraform AWS Modules

```
# infra/terraform/main.tf
module "eks" {
  source          = "terraform-aws-modules/eks/aws"
  cluster_name    = "platform-eks"
  cluster_version = "1.29"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets
  eks_managed_node_groups = {
    general = {
      instance_types = ["t3.xlarge"]
      min_size = 2; max_size = 8; desired_size = 3
    }
  }
}

module "rds" {
  source          = "terraform-aws-modules/rds/aws"
  engine          = "postgres"
  engine_version  = "16.1"
  instance_class  = "db.t3.large"
  allocated_storage = 50
  db_name         = "career_platform"
  username        = "platform"
  password        = var.db_password
  multi_az        = true
  deletion_protection = true
}

resource "aws_s3_bucket" "platform_assets" {
  bucket = "platform-assets-${var.environment}"
}

resource "aws_sqs_queue" "agent_tasks" {
  name                = "platform-agent-tasks"
  visibility_timeout_seconds = 360
  message_retention_seconds = 86400
}
```

8.3 Environment Variable Catalog

Table 8.1 Required environment variables

Variable	Service	Description
<code>DATABASE_URL</code>	All	PostgreSQL connection string
<code>REDIS_URL</code>	API, Workers	Redis connection string
<code>OPENAI_API_KEY</code>	All agents	OpenAI API key (gpt-4o, embeddings)
<code>LANGFUSE_PUBLIC_KEY</code>	All agents	Langfuse observability public key
<code>LANGFUSE_SECRET_KEY</code>	All agents	Langfuse observability secret key
<code>NEXTAUTH_SECRET</code>	Web	NextAuth.js JWT secret (min 32 chars)
<code>JWT_SECRET</code>	API	FastAPI JWT secret (must match NEXTAUTH)

<code>GOOGLE_CLIENT_ID/SECRET</code>	Web	Google OAuth credentials
<code>GITHUB_CLIENT_ID/SECRET</code>	Web	GitHub OAuth credentials
<code>LINKEDIN_CLIENT_ID/SECRET</code>	Web	LinkedIn OAuth credentials
<code>GMAIL_CLIENT_ID/SECRET</code>	API	Gmail API OAuth for email monitoring
<code>AWS_ACCESS_KEY_ID</code>	API	AWS credentials for S3 evidence storage
<code>S3_BUCKET</code>	API	Evidence and document storage bucket
<code>SLACK_WEBHOOK_URL</code>	API	Alert notifications (optional)

SECTION 09

Monitoring & Observability

Langfuse tracing, Prometheus metrics, Grafana dashboards, and alert rules

9. Monitoring & Observability

9.1 Langfuse Agent Tracing

Every LLM call is wrapped with the Langfuse handler. Traces are enriched with `userId`, `sessionId` (application_id), and `metadata` (agent_name, job_id) for full context on every span.

```
# Pattern for every agent LLM call
from langfuse.langchain import CallbackHandler

trace = langfuse.trace(
    name=f"{agent_name}_run",
    user_id=user_id,
    session_id=application_id,
    metadata={"job_id": job_id, "agent": agent_name}
)
handler = CallbackHandler(trace_id=trace.id)
response = await llm.ainvoke(prompt, config={"callbacks": [handler]})
```

9.2 Prometheus Metrics

```
# apps/api/metrics.py
from prometheus_client import Counter, Histogram, Gauge

agent_runs_total = Counter("agent_runs_total", "Total agent runs", ["agent", "status"])
agent_latency_seconds = Histogram("agent_latency_seconds", "Agent execution latency", ["agent"])
applications_total = Counter("applications_total", "Total applications submitted")
ats_score_gauge = Gauge("ats_score_current", "Current ATS score of last tailored resume")
active_jobs_gauge = Gauge("active_jobs_discovered", "Jobs pending evaluation")
token_cost_counter = Counter("llm_tokens_total", "Total LLM tokens used", ["agent", "model"])
```

9.3 Structured JSON Logging

```
# apps/api/logging_config.py - structlog configuration
import structlog

structlog.configure(
    processors=[
        structlog.contextvars.merge_contextvars,
        structlog.processors.TimeStamper(fmt="iso"),
        structlog.processors.JSONRenderer()
    ]
)

# Usage in agents:
log = structlog.get_logger()
log.info("agent_started", agent="resume_tailor", job_id=job_id, trace_id=trace_id)
```

9.4 Alert Rules

Table 9.1 Critical alert thresholds

Alert Name	Condition	Severity	Channel
HighAgentErrorRate	agent error rate > 5% over 5min	Critical	Slack + email
LowInterviewRate	interview rate < 10% over 7 days	Warning	Email
TokenCostSpike	cost/application > AUD 2.00	Warning	Slack
PendingApprovalsStale	WAITING_APPROVAL > 12 hours	Info	Email
DatabaseLatency	DB query P95 > 500ms	Critical	Slack

SECTION 10

Quick Reference

Command cheatsheet, environment checklist, and dependency versions matrix

10. Quick Reference

10.1 Command Cheatsheet

```
# — DEVELOPMENT —
pnpm install                # Install all dependencies
docker compose up -d        # Start local stack (DB, Redis)
pnpm dev                    # Start all dev servers (Turbo)
pnpm --filter @platform/web dev  # Start only Next.js frontend
pnpm --filter @platform/api dev  # Start only FastAPI backend

# — DATABASE —
pnpm prisma migrate dev     # Create and apply migration
pnpm prisma generate        # Regenerate Prisma client
pnpm prisma studio          # Open DB browser at localhost:5555
pnpm prisma db seed         # Seed with Vikram's profile data

# — TESTING —
pnpm test                   # Run all tests
pnpm test:unit              # Unit tests only
pnpm test:e2e               # Playwright E2E tests
pytest apps/api/tests/ -v   # Python tests (activate venv first)
pytest -k "test_ats" --pdb  # Run specific test with debugger

# — BUILDING —
pnpm build                  # Build all packages and apps
docker build -t platform-api ./apps/api
docker build -t platform-web ./apps/web

# — DEPLOYMENT —
terraform -chdir=infra/terraform init
terraform -chdir=infra/terraform plan
terraform -chdir=infra/terraform apply
kubectl apply -f infra/k8s/production/
kubectl rollout status deployment/platform-api -n production
kubectl logs -n production deploy/platform-api --tail=100 -f

# — AGENT DEBUGGING —
python -m agents.cli run discovery_agent --dry-run
python -m agents.cli run resume_tailor --job-id=JOB_ID --resume-id=RESUME_ID
python -m agents.cli inspect-state --thread-id=THREAD_ID
```

10.2 Dependency Versions Matrix

Table 10.1 Pinned dependency versions

Package	Version	Purpose
next	14.2.x	Frontend App Router
fastapi	0.111.x	Backend REST API
langgraph	0.2.x	Agent orchestration
langchain	0.2.x	LLM toolchain
langfuse	2.x	Agent observability
prisma	5.x	Database ORM
@prisma/client	5.x	DB client (TypeScript)

bullmq	5.x	Job queues
playwright	1.44.x	Browser automation
next-auth	4.x	Authentication
@trpc/server	11.x	Type-safe API layer
openai	1.x	OpenAI Python SDK
pydantic	2.x	Data validation
pgvector	0.3.x	Vector similarity search

10.3 Environment Setup Checklist

- Node.js 20 LTS installed (`node --version` shows v20.x)
- pnpm 8.x installed (`pnpm --version`)
- Python 3.11+ installed (`python3 --version`)
- Docker Desktop running (`docker compose ps`)
- `.env` file created from `.env.example` with all required variables
- PostgreSQL 16 + pgvector extension enabled
- Prisma migrations applied: `pnpm prisma migrate deploy`
- Langfuse project created — public + secret keys in `.env`
- OpenAI API key with gpt-4o access provisioned
- LinkedIn, Google, GitHub OAuth apps registered with correct redirect URIs
- Gmail API enabled in Google Cloud Console with OAuth consent
- AWS credentials configured for S3 bucket access
- `pnpm dev` starts without errors — all 4 services healthy
- Seed data loaded: Vikram's profile, base resume, skills visible in Prisma Studio